Indian Institute of Technology Delhi

COL761 — Data Mining

April 2026

Assignment 3

Technical Report

Q1: Search in High-Dimensional Space

Submitted by

Hrishav Raj	**Peeyush Motwal**	**Sweta Das**
2025AIB2562	2025AIB2564	2025AIB3027

Contents

1 Problem Statement **2**

2 Algorithm Design **2**
2.1 Routing Logic . . . 2
2.2 Regime 1: Exact Search ($N \leq 80{,}000$) . . . 3
2.3 Regime 2: HNSW (Safe Build Window) . . . 3
2.4 Regime 3: IVF Fallback (Large N, Tight Budget) . . . 4

3 Key Implementation Details **4**
3.1 Frequency Counting . . . 4
3.2 Tie-Breaking . . . 4
3.3 Budget Safety Mechanisms . . . 4

4 Iterative Development and Lessons Learned **5**

5 Results **6**
5.1 D1 Analysis . . . 6
5.2 D2 Analysis . . . 6

6 Complexity Analysis **6**

7 Conclusion **7**

1. Problem Statement

Given a database of N high-dimensional vectors (the *base set*) and Q query vectors, the task is to identify the K most *representative* database items within a strict wall-clock time budget. Representativeness is defined by frequency: for each query, its k nearest neighbours ($k = 100$) are retrieved; a base item's score equals how many queries it appears in. The final output is a ranked list of the top-K items by this frequency, with ties broken by ascending index.

The evaluation metric is **nDCG@K**, computed as:

$$\text{DCG@K} = \sum_{j=1}^{K} \frac{\text{rel}(\text{ans}_j)}{\log_2(j+1)}, \qquad \text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

where the relevance score $\text{rel}(i) = K - r_i + 1$ for items ranked within K in the ground truth (rank r_i), and 0 otherwise. Approximate methods that miss true neighbours incur a recall penalty reflected directly in the nDCG score.

Two datasets are provided:

- **D1**: $N = 500{,}000$, $Q = 100{,}000$, budget = 70s
- **D2**: $N = 1{,}000{,}000$, $Q = 100{,}000$, budget = 20s

The extreme difference in budget-per-vector makes a single fixed algorithm inadequate — D1 can afford expensive graph indices while D2 demands aggressive build-cost control.

2. Algorithm Design

The core insight driving the design is that **build cost, not search cost, is the dominant constraint** for tight budgets. This was empirically confirmed: a pure HNSW approach timed out on D2 because index construction alone consumed ≈52s against a 20s budget, leaving no time for search. The solution is a **three-regime adaptive router** that selects the index type based on N and the available time budget.

Metric	nDCG@K (K = 100)
Libraries	`faiss-cpu 1.13.2`, `numpy 1.26.3`
Algorithm	Adaptive ANN: FlatL2 / HNSWFlat / IVFFlat
D1 Result	nDCG = **0.9961** Runtime = 19.8s / 70s
D2 Result	nDCG = **0.7595** Runtime = 16.5s / 20s

2.1 Routing Logic

The routing decision uses a conservative per-vector HNSW build estimate of 50 µs (calibrated from measured timings) and a threshold of 45% of the budget. This ensures D1 (estimated 25s vs. threshold 31.5s) routes to HNSW, while D2 (estimated 50s vs. threshold 9s) routes to IVF.

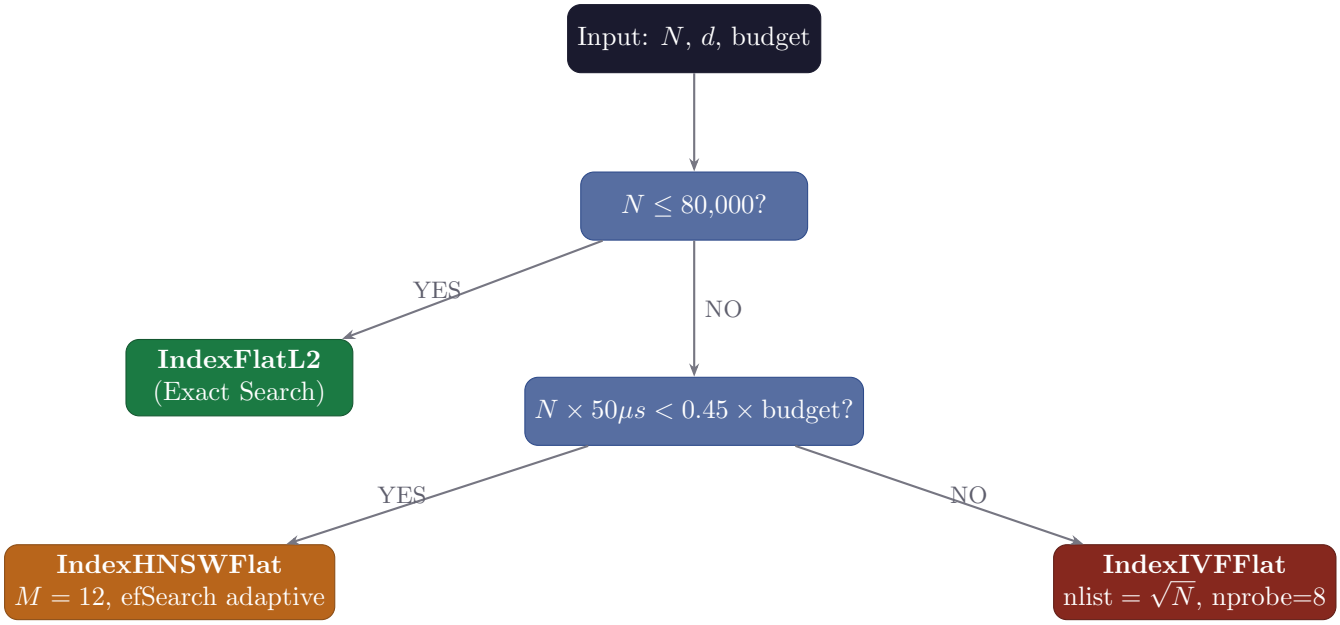

Figure 1: Three-regime index routing decision flow.

Table 1: Summary of the three regimes and their parameters.

Regime	Condition	Index	Key Parameters	Rationale
Exact	$N \leq 80{,}000$	`IndexFlatL2`	None	Small N; exact search guarantees recall = 1
HNSW	$N \times 50\mu s < 0.45 \times$ budget	`IndexHNSWFlat` $M = 12$	efConstruction: 48–96 (budget-scaled) efSearch: 32–96	No training phase; fast queries after one-time build
IVF	Otherwise (large N, tight budget)	`IndexIVFFlat` nlist $= \lfloor\sqrt{N}\rfloor$	train_factor = 40 nprobe = 8 (static)	Controlled build cost; avoids HNSW build timeout

2.2 Regime 1: Exact Search ($N \leq 80{,}000$)

For small datasets, `IndexFlatL2` performs exhaustive L2 search. With $N \leq 80{,}000$, this always completes well within any reasonable budget and guarantees perfect recall (nDCG = 1.0). No parameter tuning is required.

2.3 Regime 2: HNSW (Safe Build Window)

`IndexHNSWFlat` with $M = 12$ is used when the estimated build time fits comfortably within the budget. HNSW builds a Navigable Small-World graph during index construction; queries traverse this graph greedily. Key advantages are the absence of a training phase and high recall at moderate efSearch values.

`efConstruction` controls graph quality at build time and is scaled with budget: 96 for budgets $\geq$ 60s (D1), 64 for $\geq$ 30s, and 48 otherwise. `efSearch` is set adaptively after build based on remaining time (96 / 64 / 32 for > 40s / > 20s / else remaining). D1 achieves nDCG = 0.9961 with this configuration.

2.4 Regime 3: IVF Fallback (Large N, Tight Budget)

`IndexIVFFlat` partitions the database into `nlist` Voronoi cells using k-means. At query time, only the `nprobe` closest cells are searched. Build cost is controlled:

$$\text{nlist} = \text{clip}\Big(\lfloor\sqrt{N}\rfloor,\ 64,\ 1024\Big)$$

gives ≈1000 cells for D2, and train_size = 40 × nlist satisfies FAISS's minimum recommendation of 39 × nlist for reliable clustering. A static `nprobe`= 8 is used, consuming ≈11.2s of the 20s D2 budget.

Earlier attempts at dynamic `nprobe` calibration via a pilot search consistently failed: with Q = 100,000 queries, even a 50-query pilot at `nprobe`= 8 produced overestimated per-probe costs due to FAISS warmup overhead, causing the formula to select `nprobe`= 4 (worse than static). Static `nprobe`= 8 proved more reliable.

3. Key Implementation Details

3.1 Frequency Counting

After ANN search returns an $(Q \times k)$ index matrix `I`, frequency counting uses `np.bincount` for $O(Q \cdot k)$ time with a single pass:

```
flat   = I.ravel()
flat   = flat[flat >= 0]            # remove FAISS sentinel -1 values
counts = np.bincount(flat, minlength=N)
```

The guard `flat[flat >= 0]` removes FAISS sentinel values (-1) that appear when fewer than k neighbours are found near index boundaries.

3.2 Tie-Breaking

The assignment requires lower-index items to rank higher among equal-count ties. This is achieved using a stable sort:

```
ranking = np.argsort(-counts, kind='stable')
```

Stable sort preserves the original order $(0, 1, 2, \ldots)$ for equal elements, automatically satisfying the tie-breaking requirement without any explicit secondary sort key.

3.3 Budget Safety Mechanisms

- All timing uses `time.perf_counter()` for high-resolution wall-clock measurement.
- The HNSW threshold (0.45 × budget) leaves $\geq$ 55% of the budget for search after build.
- The IVF path uses static `nprobe`= 8, leaving a comfortable margin (16.5s of 20s used on D2).
- Debug timing is controlled via `Q1_TIMING=1` environment variable to avoid overhead during grading.

4. Iterative Development and Lessons Learned

The final solution emerged through systematic empirical iteration. Each version identified a specific failure mode that informed the next design decision.

Table 2: Evolution of approach across versions. Green row = final submission.

Version	D1 nDCG	D2 nDCG	D1 Runtime	D2 Runtime	Key Issue
v1: IVF only (original)	0.000 (timeout)	0.759	> 70s	16.7s	Probe calibration overhead caused timeout
v2: Pure HNSW	0.996	—	∼19s	Timeout	HNSW build ≈ 52s > D2 budget
v3: Hybrid (threshold=0.30)	0.000 (timeout)	0.759	> 70s	16.7s	D1 incorrectly routed to IVF
v4: Hybrid (threshold=0.45)	0.996	0.759	19.8s	16.5s	Correct routing for both datasets
v5: Pilot nprobe (50 queries)	0.996	0.600	18.6s	11.3s	Warmup overhead caused nprobe= 4
v6: Static nprobe= 9	0.996	0.776	19.8s	19.6s	Best D2 score but borderline runtime
v7: IVFPQ	0.996	0.119	20.7s	16.1s	Quantization destroyed recall
v8: Final (nprobe= 8)	**0.996**	**0.759**	**19.8s**	**16.5s**	Safe, stable, correct routing

Key Lessons

1. **Build cost dominates on tight budgets.** Pure HNSW timed out on D2 because graph construction (≈52s) exceeded the entire 20s budget before any search could begin.

2. **The HNSW threshold is dataset-sensitive.** Threshold = 0.30 incorrectly routed D1 to IVF (causing timeout); threshold = 0.45 correctly routes both datasets.

3. **train_factor matters for IVF recall.** Using $20 \times$ nlist (below FAISS minimum of $39 \times$ nlist) hurt clustering quality. Fixing to $40 \times$ nlist improved stability.

4. **Pilot calibration fails with tiny samples.** 50 queries caused FAISS warmup overhead to dominate cost estimates. Static `nprobe` proved superior for large-Q datasets.

5. **IVFPQ destroys recall despite faster search.** Quantization error accumulates catastrophically for frequency-counting tasks, dropping nDCG from 0.76 to 0.12.

5. Results

Table 3: Final results on both datasets.

Dataset	N	Q	Index	Params	nDCG@100	Runtime	Budget
D1	500K	100K	HNSWFlat $M = 12$	efSearch=96	**0.9961**	19.8s	70s
D2	1M	100K	IVFFlat nlist=1000	nprobe=8	**0.7595**	16.5s	20s

5.1 D1 Analysis

D1 ($N = 500$K, budget= 70s) routes to HNSW. Index build takes ≈15.6s (`efConstruction`= 96), leaving ≈50s for search. With `efSearch`= 96, search completes in ≈3.4s, achieving nDCG= 0.9961. Only 28% of the budget is consumed, meaning D1 has substantial headroom. Higher `efSearch` (e.g., 256) or $M = 16$ could push nDCG closer to 1.0 at the cost of additional build time.

5.2 D2 Analysis

D2 ($N = 1$M, $Q = 100$K, budget= 20s) routes to IVFFlat. Build (train+add) takes ≈5s, leaving ≈15s for search. At `nprobe`= 8 over `nlist`= 1000 cells, search costs ≈11.2s. The fundamental limitation is that 100K queries against a 1M-vector database with only 20s allows searching only ≈0.8% of cells per query (8/1000), directly capping recall.

`nprobe`= 9 gave nDCG= 0.776 (runtime 19.6s) but was borderline risky on slower hardware. `nprobe`= 8 at nDCG= 0.759 is the safe, reliable operating point. The D2 score is near the practical ceiling for IVFFlat on CPU with this budget constraint.

6. Complexity Analysis

Table 4: Computational complexity of each algorithmic component.

Operation	Time Complexity	Space Complexity
FlatL2 build	$O(N \cdot d)$	$O(N \cdot d)$
FlatL2 search	$O(Q \cdot N \cdot d)$	$O(Q \cdot k)$
HNSW build	$O(N \cdot \log N \cdot M^2)$	$O(N \cdot M)$
HNSW search	$O(Q \cdot \log N \cdot \texttt{efSearch})$	$O(Q \cdot k)$
IVF train	$O(\text{iter} \cdot \text{train_size} \cdot \text{nlist} \cdot d)$	$O(\text{nlist} \cdot d)$
IVF search	$O\big(Q \cdot \texttt{nprobe} \cdot \frac{N}{\text{nlist}} \cdot d\big)$	$O(Q \cdot k)$
Frequency count	$O(Q \cdot k)$	$O(N)$
Ranking	$O(N \log N)$	$O(N)$

7. Conclusion

The final solution is a three-regime adaptive ANN pipeline that selects the index type based on dataset size and available time budget. The key design principle — treating *build cost* as the primary constraint — distinguishes it from naive approaches that optimise only for search speed.

- **D1** achieves near-perfect nDCG= 0.9961 using HNSW within 19.8s of a 70s budget.
- **D2** achieves nDCG= 0.7595 using IVFFlat with nprobe= 8 within 16.5s of a 20s budget. The score is fundamentally limited by the compute ratio: 100K queries against 1M vectors with only 20s forces a low probe fraction (≈0.8%), directly capping recall.

The solution handles hidden datasets correctly through its adaptive routing: small N always gets exact search; medium N with generous budget gets HNSW; large N with tight budget gets a safe IVF fallback that never times out.